Considering the random factor in this algorithm, however, this is clearly still a naïve search over the possible values of T_2, and it could possibly lead to an infinite loop: to correct this situation, we could add a stop condition (and an argument) checking that a maximum number of iterations is not exceeded; at the same time, we could also store the result that is closest to our request in a temporary variable that we update at each run, and return this result whenever the maximum number of iterations is reached without finding a set of canopy with exactly `numCentroids` entries: most of the times it can be acceptable if we return, for instance, 101 centroids instead of 100 (the point being, the caller will anyway have the chance to check the result and decide).

We leave it to the readers, as an exercise, to extend listing 13.3 in order to handle the thresholds choice automatically, and instead move on to describe the distributed version of the full canopy clustering distributed algorithm, the one returning not only the canopy centroids, but also the (overlapping) sets of points that are associated with each canopy.

13.3.3 MapReduce Canopy Clustering

The *classification* step, assigning each point to one or more canopies, can be implemented in a few different ways:

- As a follow-up of the method described in listing 13.3; however, since classification will involve all points, if we don’t distribute this step, we will lose most of the advantage of running the algorithm to choose centroids in parallel;